

```
from google.colab import files
```

```
uploaded = files.upload()
```

[Choose Files](#) segmentation_task.zip

segmentation_task.zip(application/x-zip-compressed) - 123235581 bytes, last modified: 8/29/2026 - 100% done
Saving segmentation_task.zip to segmentation_task.zip

```
import zipfile
```

```
import os
```

```
zip_path = "/content/segmentation_task.zip"
```

```
extract_path = "/content/"
```

```
with zipfile.ZipFile(zip_path, "r") as zip_ref:
```

```
    zip_ref.extractall(extract_path)
```

```
print("Dataset extracted successfully!")
```

Dataset extracted successfully!

```
import os
```

```
for root, dirs, files in os.walk("/content/segmentation_task"):
```

```
    print(root, "→", len(files), "files")
```

```
/content/segmentation_task → 0 files
```

```
/content/segmentation_task/test → 0 files
```

```
/content/segmentation_task/test/masks → 860 files
```

```
/content/segmentation_task/test/images → 860 files
```

```
/content/segmentation_task/train → 0 files
```

```
/content/segmentation_task/train/masks → 3933 files
```

```
/content/segmentation_task/train/images → 3933 files
```

```
# =====  
# CELL 1 – SETUP  
# =====
```

```
import os
```

```
import glob
```

```
import random
```

```
import time
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from PIL import Image
```

```
import torch
```

```
import torch.nn as nn
```

```
import torch.nn.functional as F
```

```
from torch.utils.data import Dataset, DataLoader, random_split
```

```
import torchvision.transforms.functional as TF
```

```
print("PyTorch version:", torch.__version__)
```

```
print("CUDA available:", torch.cuda.is_available())
```

```
if torch.cuda.is_available():
```

```
    print("GPU:", torch.cuda.get_device_name(0))
```

```
    print("CUDA version:", torch.version.cuda)
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
print("Using device:", device)
```

```
# Reproducibility
```

```
SEED = 42
```

```
random.seed(SEED)
```

```
np.random.seed(SEED)
```

```
torch.manual_seed(SEED)
```

```
if torch.cuda.is_available():
```

```
    torch.cuda.manual_seed_all(SEED)
```

```
PyTorch version: 2.11.0+cu128
CUDA available: True
GPU: Tesla T4
CUDA version: 12.8
Using device: cuda
```

```
# =====
# CELL 2 - DATASET PATH
# =====

DATASET_DIR = "/content/segmentation_task"

TRAIN_IMAGE_DIR = os.path.join(DATASET_DIR, "train", "images")
TRAIN_MASK_DIR = os.path.join(DATASET_DIR, "train", "masks")

TEST_IMAGE_DIR = os.path.join(DATASET_DIR, "test", "images")
TEST_MASK_DIR = os.path.join(DATASET_DIR, "test", "masks")

print("Train images:", TRAIN_IMAGE_DIR)
print("Train masks :", TRAIN_MASK_DIR)
print("Test images :", TEST_IMAGE_DIR)
print("Test masks  :", TEST_MASK_DIR)

for path in [
    TRAIN_IMAGE_DIR,
    TRAIN_MASK_DIR,
    TEST_IMAGE_DIR,
    TEST_MASK_DIR
]:
    print(path, "→", os.path.exists(path))
```

```
Train images: /content/segmentation_task/train/images
Train masks : /content/segmentation_task/train/masks
Test images : /content/segmentation_task/test/images
Test masks  : /content/segmentation_task/test/masks
/content/segmentation_task/train/images → True
/content/segmentation_task/train/masks → True
/content/segmentation_task/test/images → True
/content/segmentation_task/test/masks → True
```

```
# =====
# CELL 3 - FIND DATASET FILES
# =====

IMAGE_EXTENSIONS = [
    "*.png",
    "*.jpg",
    "*.jpeg",
    "*.bmp",
    "*.tif",
    "*.tiff"
]

def get_files(folder):
    files = []

    for ext in IMAGE_EXTENSIONS:
        files.extend(glob.glob(os.path.join(folder, ext)))
        files.extend(glob.glob(os.path.join(folder, ext.upper())))

    return sorted(files)

train_images = get_files(TRAIN_IMAGE_DIR)
train_masks = get_files(TRAIN_MASK_DIR)

test_images = get_files(TEST_IMAGE_DIR)
test_masks = get_files(TEST_MASK_DIR)

print("Number of training images:", len(train_images))
print("Number of training masks :", len(train_masks))
print("Number of testing images :", len(test_images))
print("Number of testing masks  :", len(test_masks))
```

```
Number of training images: 3933
Number of training masks : 3933
```

Number of testing images : 860
 Number of testing masks : 860

```
# =====
# CELL 4 – VERIFY IMAGE / MASK PAIRS
# =====

def filename_stem(path):
    return os.path.splitext(os.path.basename(path))[0].lower()

train_image_dict = {
    filename_stem(x): x for x in train_images
}

train_mask_dict = {
    filename_stem(x): x for x in train_masks
}

test_image_dict = {
    filename_stem(x): x for x in test_images
}

test_mask_dict = {
    filename_stem(x): x for x in test_masks
}

train_common = sorted(
    set(train_image_dict.keys()) &
    set(train_mask_dict.keys())
)

test_common = sorted(
    set(test_image_dict.keys()) &
    set(test_mask_dict.keys())
)

train_missing_masks = sorted(
    set(train_image_dict.keys()) -
    set(train_mask_dict.keys())
)

train_missing_images = sorted(
    set(train_mask_dict.keys()) -
    set(train_image_dict.keys())
)

test_missing_masks = sorted(
    set(test_image_dict.keys()) -
    set(test_mask_dict.keys())
)

test_missing_images = sorted(
    set(test_mask_dict.keys()) -
    set(test_image_dict.keys())
)

print("===== TRAIN =====")
print("Images:", len(train_images))
print("Masks :", len(train_masks))
print("Pairs :", len(train_common))
print("Images without masks:", len(train_missing_masks))
print("Masks without images:", len(train_missing_images))

print("\n===== TEST =====")
print("Images:", len(test_images))
print("Masks :", len(test_masks))
print("Pairs :", len(test_common))
print("Images without masks:", len(test_missing_masks))
print("Masks without images:", len(test_missing_images))
```

```
===== TRAIN =====
Images: 3933
Masks : 3933
```

```
Pairs : 3933
Images without masks: 0
Masks without images: 0

===== TEST =====
Images: 860
Masks : 860
Pairs : 860
Images without masks: 0
Masks without images: 0
```

```
# =====
# CELL 5 – CREATE IMAGE/MASK PAIRS
# =====

train_pairs = [
    (train_image_dict[name], train_mask_dict[name])
    for name in train_common
]

test_pairs = [
    (test_image_dict[name], test_mask_dict[name])
    for name in test_common
]

print("Training pairs:", len(train_pairs))
print("Testing pairs :", len(test_pairs))

print("\nExample:")
print(train_pairs[0])
```

```
Training pairs: 3933
Testing pairs : 860
```

```
Example:
('/content/segmentation_task/train/images/brisc2025_train_00001_g1_ax_t1.jpg', '/content/segmentation_task/train/masks/brisc20
```

```
# =====
# CELL 6 – INSPECT ONE IMAGE AND MASK
# =====

image_path, mask_path = train_pairs[0]

image = Image.open(image_path)
mask = Image.open(mask_path)

print("Image:", image.size, image.mode)
print("Mask :", mask.size, mask.mode)

print("Image array shape:", np.array(image).shape)
print("Mask array shape :", np.array(mask).shape)

print("Mask unique values:")
print(np.unique(np.array(mask))[:30])
```

```
Image: (512, 512) RGB
Mask : (512, 512) L
Image array shape: (512, 512, 3)
Mask array shape : (512, 512)
Mask unique values:
[ 0  1  2  3  4  5  6  7 248 249 250 251 252 253 254 255]
```

```
# =====
# CELL 7 – VISUALIZE SAMPLE
# =====

def show_sample(image_path, mask_path):

    image = Image.open(image_path).convert("RGB")
    mask = Image.open(mask_path).convert("L")

    image_np = np.array(image)
    mask_np = np.array(mask)

    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
```

```

axes[0].imshow(image_np)
axes[0].set_title("Original MRI")
axes[0].axis("off")

axes[1].imshow(mask_np, cmap="gray")
axes[1].set_title("Ground Truth Mask")
axes[1].axis("off")

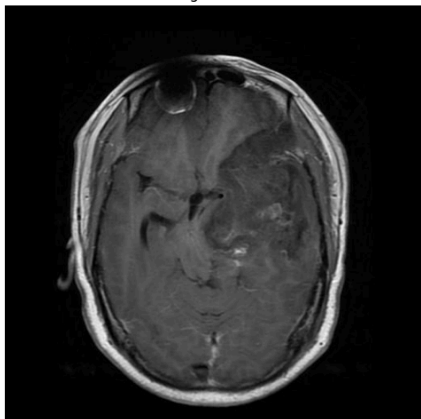
axes[2].imshow(image_np)
axes[2].imshow(mask_np, cmap="jet", alpha=0.4)
axes[2].set_title("MRI + Mask")
axes[2].axis("off")

plt.tight_layout()
plt.show()

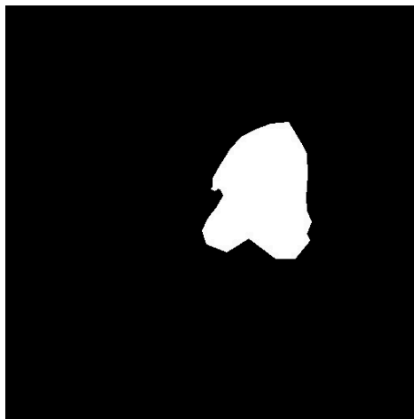
show_sample(*train_pairs[0])

```

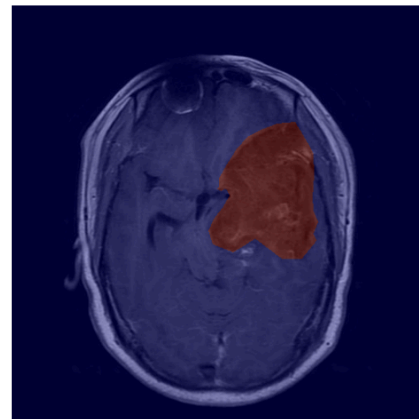
Original MRI



Ground Truth Mask



MRI + Mask



```

# =====
# CELL 8 - MULTIPLE SAMPLES
# =====

def show_multiple_samples(pairs, n=6):

    samples = random.sample(
        pairs,
        min(n, len(pairs))
    )

    fig, axes = plt.subplots(
        len(samples),
        3,
        figsize=(12, 4 * len(samples))
    )

    if len(samples) == 1:
        axes = np.expand_dims(axes, axis=0)

    for row, (image_path, mask_path) in enumerate(samples):

        image = np.array(
            Image.open(image_path).convert("RGB")
        )

        mask = np.array(
            Image.open(mask_path).convert("L")
        )

        axes[row, 0].imshow(image)
        axes[row, 0].set_title("MRI")

        axes[row, 1].imshow(mask, cmap="gray")
        axes[row, 1].set_title("Ground Truth")

        axes[row, 2].imshow(image)
        axes[row, 2].imshow(mask, cmap="jet", alpha=0.4)

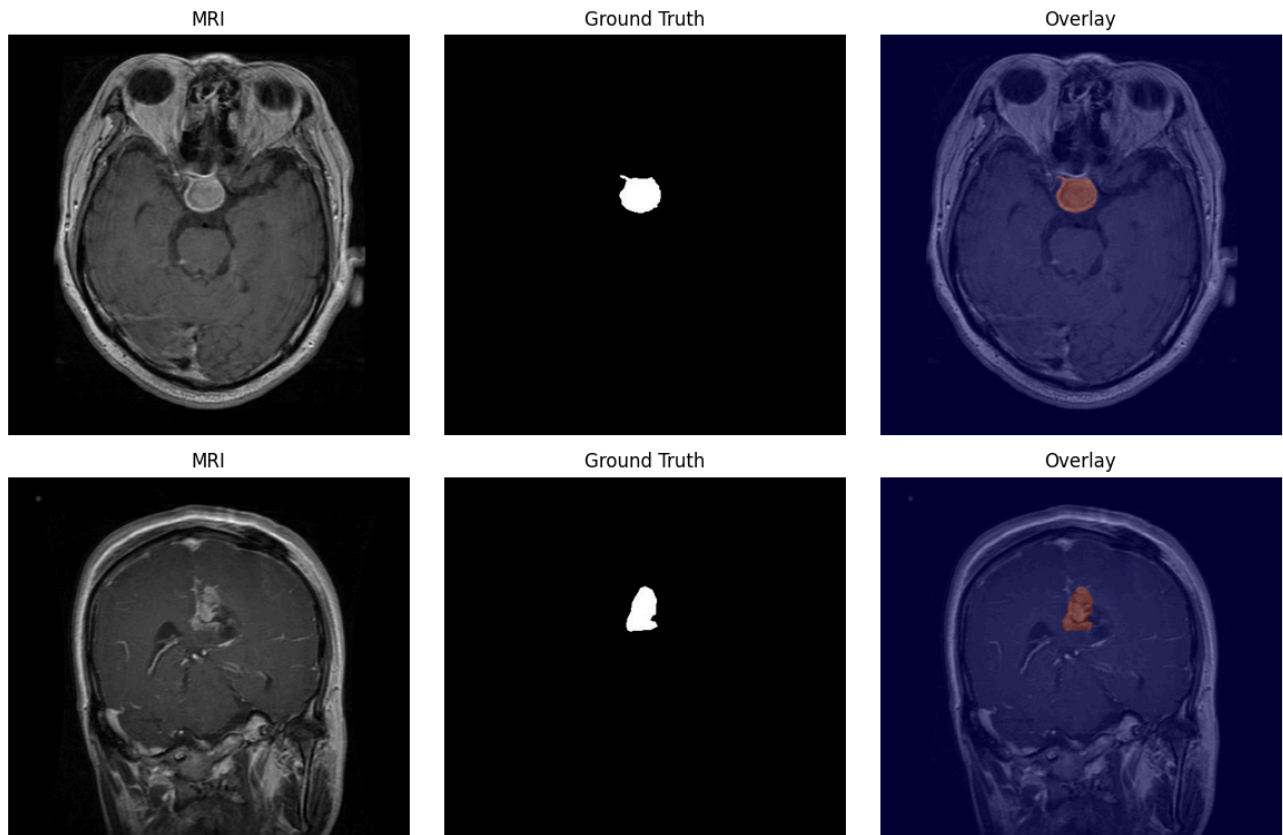
```

```
axes[row, 2].imshow(mask, cmap=jet, alpha=0.4)
axes[row, 2].set_title("Overlay")

for col in range(3):
    axes[row, col].axis("off")

plt.tight_layout()
plt.show()

show_multiple_samples(train_pairs, n=6)
```

```
# =====
# CELL 9 - CONFIGURATION
# =====

IMG_SIZE = 224
BATCH_SIZE = 16

NUM_EPOCHS = 15
LEARNING_RATE = 1e-4

EARLY_STOPPING_PATIENCE = 3

print("Image size:", IMG_SIZE)
print("Batch size:", BATCH_SIZE)
print("Epochs:", NUM_EPOCHS)
print("Learning rate:", LEARNING_RATE)
```

Image size: 224

Batch size: 16

Epochs: 15

Learning rate: 0.0001

```
# =====
# CELL 10 - PYTORCH SEGMENTATION DATASET
# =====

class BrainTumorSegmentationDataset(Dataset):

    def __init__(self, pairs, img_size=224):

        self.pairs = pairs
        self.img_size = img_size

    def __len__(self):
        return len(self.pairs)

    def __getitem__(self, idx):

        image_path, mask_path = self.pairs[idx]

        # Load image
        image = Image.open(image_path).convert("RGB")
```



```

# Load mask
mask = Image.open(mask_path).convert("L")

# Resize
image = image.resize(
    (self.img_size, self.img_size),
    Image.Resampling.BILINEAR
)

# IMPORTANT:
# Masks use NEAREST interpolation
mask = mask.resize(
    (self.img_size, self.img_size),
    Image.Resampling.NEAREST
)

# Convert to numpy
image = np.array(image).astype(np.float32)
mask = np.array(mask).astype(np.float32)

# Normalize image
image = image / 255.0

# Convert mask to binary
mask = (mask > 0).astype(np.float32)

# HWC → CHW
image = torch.from_numpy(
    image.transpose(2, 0, 1)
).float()

mask = torch.from_numpy(mask).unsqueeze(0).float()

return image, mask

```

```

# =====
# CELL 11 – TRAIN / VALIDATION SPLIT
# =====

random.shuffle(train_pairs)

total_train = len(train_pairs)

val_size = int(0.15 * total_train)
train_size = total_train - val_size

train_split_pairs = train_pairs[:train_size]
val_split_pairs = train_pairs[train_size:]

print("Training samples :", len(train_split_pairs))
print("Validation samples:", len(val_split_pairs))
print("Testing samples  :", len(test_pairs))

```

```

Training samples : 3344
Validation samples: 589
Testing samples  : 860

```

```

# =====
# CELL 12 – CREATE DATASETS
# =====

train_dataset = BrainTumorSegmentationDataset(
    train_split_pairs,
    img_size=IMG_SIZE
)

val_dataset = BrainTumorSegmentationDataset(
    val_split_pairs,
    img_size=IMG_SIZE
)

test_dataset = BrainTumorSegmentationDataset(
    test_pairs,
    img_size=IMG_SIZE
)

```

```
)

print("Datasets created.")
```

Datasets created.

```
# =====
# CELL 13 - DATALOADERS
# =====

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=2,
    pin_memory=True,
    persistent_workers=True
)

val_loader = DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=2,
    pin_memory=True,
    persistent_workers=True
)

test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=2,
    pin_memory=True,
    persistent_workers=True
)

print("Train batches:", len(train_loader))
print("Val batches  :", len(val_loader))
print("Test batches :", len(test_loader))
```

Train batches: 209
Val batches : 37
Test batches : 54

```
# =====
# CELL 14 - VERIFY BATCH
# =====

images, masks = next(iter(train_loader))

print("Image batch shape:", images.shape)
print("Mask batch shape :", masks.shape)

print("Image range:", images.min().item(), "to", images.max().item())
print("Mask range :", masks.min().item(), "to", masks.max().item())
```

Image batch shape: torch.Size([16, 3, 224, 224])
Mask batch shape : torch.Size([16, 1, 224, 224])
Image range: 0.0 to 1.0
Mask range : 0.0 to 1.0

```
# =====
# CELL 15 - U-NET
# =====

class DoubleConv(nn.Module):

    def __init__(self, in_channels, out_channels):

        super().__init__()

        self.block = nn.Sequential(

            nn.Conv2d(
```

```

        in_channels,
        out_channels,
        kernel_size=3,
        padding=1
    ),

    nn.BatchNorm2d(out_channels),
    nn.ReLU(inplace=True),

    nn.Conv2d(
        out_channels,
        out_channels,
        kernel_size=3,
        padding=1
    ),

    nn.BatchNorm2d(out_channels),
    nn.ReLU(inplace=True)
)

def forward(self, x):
    return self.block(x)

class UNet(nn.Module):

    def __init__(self):

        super().__init__()

        # Encoder
        self.enc1 = DoubleConv(3, 64)
        self.enc2 = DoubleConv(64, 128)
        self.enc3 = DoubleConv(128, 256)
        self.enc4 = DoubleConv(256, 512)

        self.pool = nn.MaxPool2d(2)

        # Bottleneck
        self.bottleneck = DoubleConv(512, 1024)

        # Decoder
        self.up4 = nn.ConvTranspose2d(
            1024, 512, kernel_size=2, stride=2
        )
        self.dec4 = DoubleConv(1024, 512)

        self.up3 = nn.ConvTranspose2d(
            512, 256, kernel_size=2, stride=2
        )
        self.dec3 = DoubleConv(512, 256)

        self.up2 = nn.ConvTranspose2d(
            256, 128, kernel_size=2, stride=2
        )
        self.dec2 = DoubleConv(256, 128)

        self.up1 = nn.ConvTranspose2d(
            128, 64, kernel_size=2, stride=2
        )
        self.dec1 = DoubleConv(128, 64)

        # Output
        self.out = nn.Conv2d(
            64,
            1,
            kernel_size=1
        )

    def forward(self, x):

        # Encoder
        e1 = self.enc1(x)
        e2 = self.enc2(self.pool(e1))
        e3 = self.enc3(self.pool(e2))
        e4 = self.enc4(self.pool(e3))

```

```

# Bottleneck
b = self.bottleneck(self.pool(e4))

# Decoder
d4 = self.up4(b)
d4 = torch.cat([d4, e4], dim=1)
d4 = self.dec4(d4)

d3 = self.up3(d4)
d3 = torch.cat([d3, e3], dim=1)
d3 = self.dec3(d3)

d2 = self.up2(d3)
d2 = torch.cat([d2, e2], dim=1)
d2 = self.dec2(d2)

d1 = self.up1(d2)
d1 = torch.cat([d1, e1], dim=1)
d1 = self.dec1(d1)

return self.out(d1)

```

```

# =====
# CELL 16 - MODEL
# =====

model = UNet().to(device)

print(model)

total_params = sum(
    p.numel()
    for p in model.parameters()
)

print(
    f"\nTotal parameters: {total_params:}"
)

```

```

        (4): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (5): ReLU(inplace=True)
    )
)
(up1): ConvTranspose2d(128, 64, kernel_size=(2, 2), stride=(2, 2))
(dec1): DoubleConv(
  (block): Sequential(
    (0): Conv2d(128, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (5): ReLU(inplace=True)
  )
)
(out): Conv2d(64, 1, kernel_size=(1, 1), stride=(1, 1))
)

```

Total parameters: 31,043,521

```

# =====
# CELL 17 - LOSS AND METRICS
# =====

```

```
def dice_score(pred, target, smooth=1e-6):
```

```

    pred = (pred > 0.5).float()

    pred = pred.view(pred.size(0), -1)
    target = target.view(target.size(0), -1)

    intersection = (pred * target).sum(dim=1)

    dice = (
        2 * intersection + smooth
    ) / (
        pred.sum(dim=1) +
        target.sum(dim=1) +
        smooth
    )

    return dice.mean()

```

```
def iou_score(pred, target, smooth=1e-6):
```

```

    pred = (pred > 0.5).float()

    pred = pred.view(pred.size(0), -1)
    target = target.view(target.size(0), -1)

    intersection = (pred * target).sum(dim=1)

    union = (
        pred.sum(dim=1) +
        target.sum(dim=1) -
        intersection
    )

    iou = (
        intersection + smooth
    ) / (
        union + smooth
    )

    return iou.mean()

```

```
def dice_loss(logits, target, smooth=1e-6):
```

```

    probs = torch.sigmoid(logits)

    probs = probs.view(probs.size(0), -1)
    target = target.view(target.size(0), -1)

    intersection = (probs * target).sum(dim=1)

    dice = (

```

```

        2 * intersection + smooth
    ) / (
        probs.sum(dim=1) +
        target.sum(dim=1) +
        smooth
    )

    return 1 - dice.mean()

bce_loss = nn.BCEWithLogitsLoss()

def combined_loss(logits, target):

    bce = bce_loss(logits, target)
    dice = dice_loss(logits, target)

    return bce + dice

```

```

# =====
# CELL 18 - OPTIMIZER
# =====

optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=LEARNING_RATE,
    weight_decay=1e-5
)

scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode="max",
    factor=0.5,
    patience=2
)

print("Optimizer:", optimizer)

```

```

Optimizer: AdamW (
Parameter Group 0
  amsgrad: False
  betas: (0.9, 0.999)
  capturable: False
  decoupled_weight_decay: True
  differentiable: False
  eps: 1e-08
  foreach: None
  fused: None
  lr: 0.0001
  maximize: False
  weight_decay: 1e-05
)

```

```

# =====
# FAST U-NET MODEL
# =====

class DoubleConv(nn.Module):

    def __init__(self, in_channels, out_channels):
        super().__init__()

        self.block = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, 3, padding=1),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),

            nn.Conv2d(out_channels, out_channels, 3, padding=1),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
        )

    def forward(self, x):
        return self.block(x)

```

```

class FastUNet(nn.Module):

    def __init__(self):
        super().__init__()

        # Encoder
        self.enc1 = DoubleConv(3, 32)
        self.enc2 = DoubleConv(32, 64)
        self.enc3 = DoubleConv(64, 128)
        self.enc4 = DoubleConv(128, 256)

        self.pool = nn.MaxPool2d(2)

        # Bottleneck
        self.bottleneck = DoubleConv(256, 512)

        # Decoder
        self.up4 = nn.ConvTranspose2d(
            512, 256, 2, stride=2
        )
        self.dec4 = DoubleConv(512, 256)

        self.up3 = nn.ConvTranspose2d(
            256, 128, 2, stride=2
        )
        self.dec3 = DoubleConv(256, 128)

        self.up2 = nn.ConvTranspose2d(
            128, 64, 2, stride=2
        )
        self.dec2 = DoubleConv(128, 64)

        self.up1 = nn.ConvTranspose2d(
            64, 32, 2, stride=2
        )
        self.dec1 = DoubleConv(64, 32)

        # Output
        self.out = nn.Conv2d(32, 1, 1)

    def forward(self, x):

        e1 = self.enc1(x)

        e2 = self.enc2(
            self.pool(e1)
        )

        e3 = self.enc3(
            self.pool(e2)
        )

        e4 = self.enc4(
            self.pool(e3)
        )

        b = self.bottleneck(
            self.pool(e4)
        )

        d4 = self.up4(b)
        d4 = torch.cat([d4, e4], dim=1)
        d4 = self.dec4(d4)

        d3 = self.up3(d4)
        d3 = torch.cat([d3, e3], dim=1)
        d3 = self.dec3(d3)

        d2 = self.up2(d3)
        d2 = torch.cat([d2, e2], dim=1)
        d2 = self.dec2(d2)

        d1 = self.up1(d2)
        d1 = torch.cat([d1, e1], dim=1)
        d1 = self.dec1(d1)

```

```

        return self.out(d1)

print("FastUNet defined successfully!")

```

FastUNet defined successfully!

```

model = FastUNet().to(device)

optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=1e-4,
    weight_decay=1e-5
)

scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode="max",
    factor=0.5,
    patience=2
)

print("Model ready!")
print("Device:", device)

```

Model ready!
Device: cuda

```

def quick_train(
    model,
    train_loader,
    val_loader,
    optimizer,
    epochs=2
):

    scaler = torch.amp.GradScaler(
        "cuda",
        enabled=torch.cuda.is_available()
    )

    for epoch in range(epochs):

        start = time.time()

        model.train()
        total_loss = 0

        for images, masks in train_loader:

            images = images.to(
                device,
                non_blocking=True
            )

            masks = masks.to(
                device,
                non_blocking=True
            )

            optimizer.zero_grad(
                set_to_none=True
            )

            with torch.amp.autocast(
                device_type="cuda",
                enabled=torch.cuda.is_available()
            ):

                outputs = model(images)
                loss = combined_loss(outputs, masks)

            scaler.scale(loss).backward()
            scaler.step(optimizer)

```



```

        scaler.update()

        total_loss += loss.item()

    avg_loss = total_loss / len(train_loader)

    elapsed = time.time() - start

    print(
        f"Epoch {epoch+1}/{epochs} | "
        f"Loss: {avg_loss:.4f} | "
        f"Time: {elapsed:.1f}s"
    )

print("Quick training completed!")

```

```

# =====
# FIXED DATALOADERS FOR COLAB
# =====

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=0,
    pin_memory=torch.cuda.is_available()
)

val_loader = DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=0,
    pin_memory=torch.cuda.is_available()
)

test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=0,
    pin_memory=torch.cuda.is_available()
)

print("DataLoaders recreated successfully.")
print("Train batches:", len(train_loader))
print("Validation batches:", len(val_loader))
print("Test batches:", len(test_loader))

```

```

DataLoaders recreated successfully.
Train batches: 209
Validation batches: 37
Test batches: 54

```

```

# =====
# TEST DATA LOADING
# =====

images, masks = next(iter(train_loader))

print("Images:", images.shape)
print("Masks :", masks.shape)
print("Image min/max:", images.min().item(), images.max().item())
print("Mask min/max :", masks.min().item(), masks.max().item())

```

```

Images: torch.Size([16, 3, 224, 224])
Masks : torch.Size([16, 1, 224, 224])
Image min/max: 0.0 1.0
Mask min/max : 0.0 1.0

```

```

quick_train(
    model,
    train_loader,
    val_loader,
    optimizer,

```

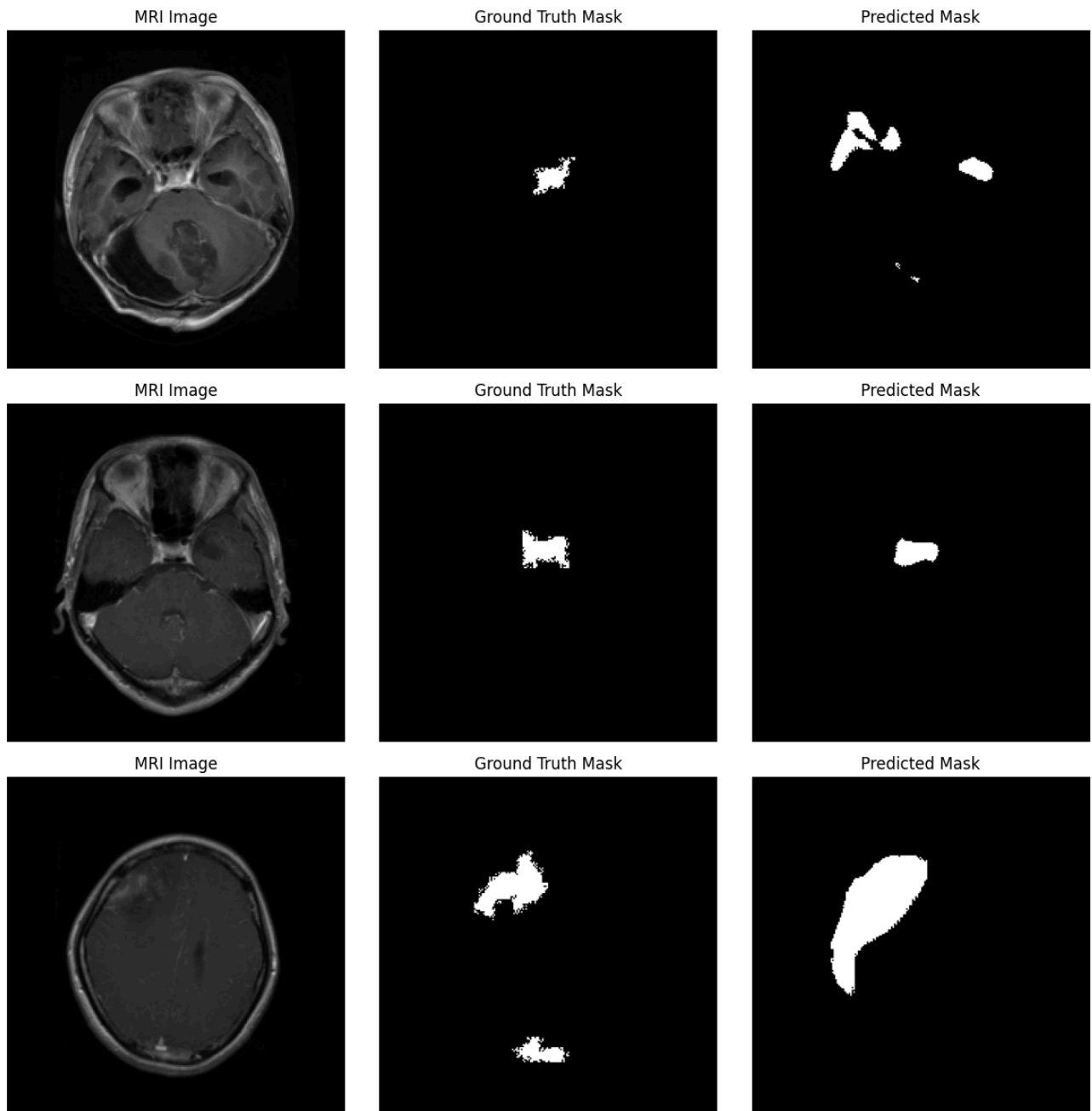
```
    epochs=2  
)
```

```
Epoch 1/2 | Loss: 1.3247 | Time: 43.5s  
Epoch 2/2 | Loss: 1.1809 | Time: 42.5s  
Quick training completed!
```

```
print("Test samples:", len(test_dataset))
```

```
Test samples: 860
```

```
import matplotlib.pyplot as plt  
import torch  
  
model.eval()  
  
fig, axes = plt.subplots(3, 3, figsize=(12, 12))  
  
with torch.no_grad():  
    for i in range(3):  
        image, mask = test_dataset[i]  
        input_image = image.unsqueeze(0).to(device)  
        output = model(input_image)  
        prediction = torch.sigmoid(output)[0, 0]  
        prediction = (prediction > 0.5).float()  
        image_display = image.permute(1, 2, 0).cpu().numpy()  
        mask_display = mask.squeeze().cpu().numpy()  
        pred_display = prediction.cpu().numpy()  
        axes[i, 0].imshow(image_display)  
        axes[i, 0].set_title("MRI Image")  
        axes[i, 0].axis("off")  
        axes[i, 1].imshow(mask_display, cmap="gray")  
        axes[i, 1].set_title("Ground Truth Mask")  
        axes[i, 1].axis("off")  
        axes[i, 2].imshow(pred_display, cmap="gray")  
        axes[i, 2].set_title("Predicted Mask")  
        axes[i, 2].axis("off")  
  
plt.tight_layout()  
plt.show()
```



```
def evaluate_segmentation(model, loader):

    model.eval()

    total_dice = 0
    total_iou = 0
    count = 0

    with torch.no_grad():

        for images, masks in loader:

            images = images.to(device)
            masks = masks.to(device)

            outputs = model(images)

            preds = (torch.sigmoid(outputs) > 0.5).float()

            intersection = (preds * masks).sum(
```